—Doug McIlroy, Basics of the Unix Philosophy[5,6]

STDIO was developed by Ken Thompson[7] as a part of the infrastructure required to implement pipes on early versions of Unix. Programs that implement STDIO use standardized file handles for input and output rather than files that are stored on a disk or other recording media. STDIO is best described as a buffered data stream, and its primary function is to stream data from the output of one program, file, or device to the input of another program, file, or device.

Data streams are the raw materials upon which the core utilities and many other CLI tools perform their work. As its name implies, a data stream is a stream of data being passed from one file, device, or program to another using STDIO.

Transforming Data Streams

This tenet explores the use of pipes to connect streams of data from one utility program to another using STDIO. The function of these programs is to transform the data in some manner. You will also learn about the use of redirection to redirect the data to a file.

Data streams can be manipulated by using pipes to insert transformers into the stream. Each transformer program is used by the SysAdmin to perform some transformational operation on the data in the stream, thus changing its contents in some manner. Redirection can then be used at the end of the pipeline to direct the data stream to a file. As has already been mentioned, that file could be an actual data file on the hard drive or a device file such as a drive partition, a printer, a terminal, a pseudo-terminal, or any other device connected to a computer.

I use the term "transform" in conjunction with these programs because the primary task of each is to transform the incoming data from STDIO in a specific way as intended by the SysAdmin and to send the transformed data to STDOUT for possible use by another transformer program or redirection to a file.

The standard term for these programs, "filters," implies something with which I don't always agree. By definition, a filter is a device or a tool that removes something, such as an air filter removing airborne contaminants so that the internal combustion engine of your automobile does not grind itself to death on those particulates. In my high school and college chemistry classes, filter paper was used to remove particulates from a liquid. The air filter in my home HVAC system removes particulates that I don't want to breathe. So, although they do sometimes filter out unwanted data from a stream, I much prefer the term "transformers" because these utilities do so much more. They can add data to a stream, modify the data in some amazing ways, sort it, rearrange the data in each line, perform operations based on the contents of the data stream, and so much more. Feel free to use whichever term you prefer, but I prefer transformers.

The ability to manipulate these data streams using these small yet powerful transformer programs is central to the power of the Linux command-line interface. Many of the Linux core utilities are transformer programs and use STDIO.

Everything Is a File

This is one of the most important concepts that makes Linux especially flexible and powerful: everything is a file. That is, everything can be the source of a data stream, the target of a data stream, or in many cases both. In this course you will explore what "everything is a file" really means and learn to use that to your great advantage as a SysAdmin.

The whole point with "everything is a file" is... the fact that you can use common tools to operate on different things.
—Linus Torvalds in an email

The idea that everything is a file has some interesting and amazing implications. This concept makes it possible to copy a boot record, a disk partition, or an entire hard drive including the boot record, because the entire hard drive is a file, just as are the individual partitions. Other possibilities include using the **cp** (copy) command to print a PDF file to a compatible printer, using the **echo** command to send messages from one terminal session to another, and using the **dd** command to copy ISO image files to a USB thumb drive.

"Everything is a file" is possible because all devices are implemented by Linux as these things called device special files, which are located in the /dev/ directory. Device files are not device drivers; rather, they are gateways to devices that are exposed to the user. We will discuss device special files in some detail throughout this course as well as in Volume 2, Chapter 22.

Use the Linux FHS

The Linux Filesystem Hierarchical Standard (FHS) defines the structure of the Linux directory tree. It names a set of standard directories and designates their purposes. This standard has been put in place to ensure that all distributions of Linux are consistent in their directory usage. Such consistency makes writing and maintaining shell and compiled programs easier for SysAdmins because the programs, their configuration files, and their data, if any, should be located in the standard directories. This tenet is about storing programs and data in the standard and recommended locations in the directory tree and the advantages of doing so.

As SysAdmins our tasks include everything from fixing problems to writing CLI programs to perform many of our tasks for us and for others. Knowing where data of various types are intended to be stored on a Linux system can be very helpful in resolving problems as well as preventing them.

The latest Filesystem Hierarchical Standard (3.0)[8] is defined in a document maintained by the Linux Foundation.[9] The document is available in multiple formats from their website as are historical versions of the FHS.

Embrace the CLI

The Force is with Linux and the Force is the command-line interface – the CLI. The vast power of the Linux CLI lies in its complete lack of restrictions. Linux provides many options for accessing the command line such as virtual consoles, many different terminal emulators, shells, and other related software that can enhance your flexibility and productivity.

The command line is a tool that provides a text-mode interface between the user and the operating system. The command line allows the user to type commands into the computer for processing and to see the results.

The Linux command-line interface is implemented with shells such as bash (Bourne again shell), csh (C shell), and ksh (Korn shell) to name just three of the many that are available. The function of any shell is to pass commands typed by the user to the operating system, which executes the commands and returns the results to the shell.

Access to the command line is through a terminal interface of some type. There are three primary types of terminal interface that are common in modern Linux computers, but the terminology can be confusing. These three interfaces are virtual consoles, terminal emulators that run on a graphical desktop, and an SSH remote connection. We will explore the terminology, virtual consoles, and one terminal emulator in Chapter 7. We look at several different terminal emulators in Chapter 14.

Be the Lazy SysAdmin

Despite everything we were told by our parents, teachers, bosses, well-meaning authority figures, and hundreds of quotes about hard work that I found with a Google search, getting your work done well and on time is not the same as working hard. One does not necessarily imply the other.

I am a lazy SysAdmin. I am also a very productive SysAdmin. Those two seemingly contradictory statements are not mutually exclusive; rather, they are complementary in a very positive way. Efficiency is the only way to make this possible.

This tenet is about working hard at the right tasks to optimize our own efficiency as SysAdmins. Part of this is about automation, which we will explore in detail in Chapter 29 of Volume 2, but also throughout this course. The greater part of this tenet is about finding many of the myriad ways to use the shortcuts already built into Linux.

These are things like using aliases as shortcuts to reduce typing – but probably not in the way you think of them if you come from a Windows background. Naming files so that they can be easily found in lists, using the file name completion facility that is part of bash, the default Linux shell for most distributions, and more all contribute to making life easier for lazy SysAdmins.

Automate Everything

The function of computers is to automate mundane tasks in order to allow us humans to concentrate on the tasks that the computers cannot – yet – do. For SysAdmins, those of us who run and manage the computers most closely, we have direct access to the tools that can help us work more efficiently. We should use those tools to maximum benefit.

In Chapter 8 of *The Linux Philosophy for SysAdmins*,[10] I state, "A SysAdmin is most productive when thinking – thinking about how to solve existing problems and about how to avoid future problems; thinking about how to monitor Linux computers in order to find clues that anticipate and foreshadow those future problems; thinking about how to make their job more efficient; thinking about how to automate all of those tasks that need to be performed whether every day or once a year."

SysAdmins are next most productive when creating the shell programs that automate the solutions that they have conceived while appearing to be unproductive. The more automation we have in place, the more time we have available to fix real problems when they occur and to contemplate how to automate even more than we already have.

D. Both, *Using and Administering Linux: Volume 1*
https://doi.org/10.1007/978-1-4842-9618-9_9

9. Data Streams

David Both[1] ✉

(1) Raleigh, NC, USA

Objectives

In this chapter you will learn

- How text data streams form the architectural basis for the extreme flexibility of the Linux command line
- How to generate streams of text data
- How to create a new logical volume (LV) for use in the experiments in this chapter
- How to use pipes, STDIO, and many of the core utilities to manipulate text data streams
- How to redirect data streams to and from files
- Basic usage of some of the special device files in the /dev directory

Data Streams as Raw Materials

Everything in Linux revolves around streams of data – particularly text streams.

Data streams are the raw materials upon which the core utilities and many other CLI tools perform their work. As its name implies, a data stream is a stream of data – text data – being passed from one file, device, or program to another using Standard Input/Output (STDIO). This chapter introduces the use of pipes to connect streams of data from one filter program to another using STDIO. You will learn that the function of these programs is to transform the data in some manner. You will also learn about the use of redirection to redirect the data to a file. A filter is defined as

> *A program that processes an input data stream into an output data stream in some well-defined way, and does no I/O to anywhere else except possibly on error conditions; one designed to be used as a stage in a pipeline.*
> —The Free On-line Dictionary of Computing (FOLDOC)

Data streams can be manipulated by inserting one or more filter programs into the stream using pipes. Each filter is used by the SysAdmin to perform some operation on the data in the stream, thus changing its contents in some well-defined manner. Redirection can then be used at the end of the pipeline to direct the data stream to a file. As has already been mentioned, that file could be an actual data file on the hard drive or a device file such as a drive partition, a printer, a terminal, a pseudo-terminal, or any other device[1] connected to a computer.

The ability to manipulate these data streams using these small yet powerful filters is central to the power of the Linux command-line interface. Many of the core utilities are filter programs and use STDIO.

I recently Googled "data stream," and most of the top hits are concerned with processing huge amounts of streaming data in single entities such as streaming video and audio or financial institutions processing streams consisting of huge numbers of individual transactions. This is not what we are talking about here although the concept is the same, and a case could be made that current applications use the stream processing functions of Linux as the model for processing many types of data.

In the Linux world, a stream is a flow of text data that originates at some source; the stream may flow to one or more programs that transform it in some way, and then it may be stored in a file or displayed in a terminal session. As a SysAdmin your job is intimately associated with manipulating the creation and flow of these data streams. In this chapter we will explore data streams – what they are, how to create them, and a little bit about how to use them.

Text Streams: A Universal Interface

The use of Standard Input/Output (STDIO) for program input and output is a key foundation of the Linux way of doing things. STDIO was first developed for Unix and has found its way into most other operating systems since then, including DOS, Windows, and Linux.

> *This is the Unix philosophy: Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface.*
> —Doug McIlroy, Basics of the Unix Philosophy[2,3]

STDIO was developed by Ken Thompson[4] as a part of the infrastructure required to implement pipes on early versions of Unix. Programs that implement STDIO use standardized file handles for input and output rather than files that are stored on a disk or other recording media. STDIO is best described as a buffered data stream, and its primary function is to stream data from the output of one program, file, or device to the input of another program, file, or device.

STDIO File Handles

There are three STDIO data streams, each of which is automatically opened as a file at the startup of a program – well, those programs that use STDIO. Each STDIO data stream is associated with a file handle, which is just a set of metadata that describes the attributes of the file. File handles 0, 1, and 2 are explicitly defined by convention and long practice as STDIN, STDOUT, and STDERR, respectively.

STDIN, file handle 0, is standard input, which is usually input from the keyboard. STDIN can be redirected from any file including device files instead of the keyboard. It is not common to need to redirect STDIN, but it can be done.
STDOUT, file handle 1, is standard output, which sends the data stream to the terminal by default. It is common to redirect STDOUT to a file or to pipe it to another program for further processing.
STDERR is associated with file handle 2. The data stream for STDERR is also usually sent to the terminal.

If STDOUT is redirected to a file, STDERR continues to be displayed on the screen. This ensures that when the data stream itself is not displayed on the terminal, that STDERR is, thus, ensuring that the user will see any errors resulting from execution of the program. STDERR can also be redirected to the same or passed on to the next filter program in a pipeline.

STDIO is implemented in a standard C library header file, stdio.h, which can be included in the source code of programs so that it can be compiled into the resulting executable.

Preparing a Logical Volume for Testing

Many of the experiments in this chapter can be dangerous when performed on the logical volumes that have already been created during the installation. Therefore, you will create a new LV (logical volume) for use in testing. You can perform the following experiments safely with a test volume that is not being used for anything else.

Performing this and other tasks on logical volumes and volume groups is a very typical task for SysAdmins. I do things like this quite frequently. We will explore logical volume management in detail in Volume 2, Chapter 20.

Storage Device Logical Structure

Let's take a little side trip so you can get a better understanding of what you will be doing in this section. Look again at the storage devices on your VM. We are especially interested in the tree belonging to the /dev/sda device:

```
[root@studentvm1 ~]# lsblk
NAME                        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                           8:0    0   60G  0 disk
├─sda1                        8:1    0    1M  0 part
├─sda2                        8:2    0    1G  0 part /boot
├─sda3                        8:3    0    1G  0 part /boot/efi
└─sda4                        8:4    0   58G  0 part
  ├─fedora_studentvm1-root 253:0    0    2G  0 lvm  /
  ├─fedora_studentvm1-usr  253:1    0   15G  0 lvm  /usr
  ├─fedora_studentvm1-tmp  253:2    0    5G  0 lvm  /tmp
  ├─fedora_studentvm1-var  253:3    0   10G  0 lvm  /var
  └─fedora_studentvm1-home 253:4    0    2G  0 lvm  /home
  sr0                       11:0    1 50.5M  0 rom
zram0                      252:0    0    8G  0 disk [SWAP]
```

This tree illustrates the logical meta-structure created on the storage device during the installation of Fedora. This structure was created from the partitioning and LVM specifications you entered on the Manual Partitioning page of the Anaconda installer.

The Type column shows the kind of meta-structure for each of the devices in the tree. The sda device is the ID for the disk device as a whole. sda1 through sda3 are partitions created for files related to bootup. The sda1 partition is the BIOS boot partition. The other two boot partitions are obvious.

All these partitions are standard Linux Type 83 partitions.[5] The sda4 partition has then been configured as a physical volume (PV) for logical volume management (LVM).[6] The entire PV was then configured as a volume group (VG) that was named fedora_studentvm1 by the installer. The various logical volumes (LV) were created in the VG.

The sr0 partition is the (virtual) DVD device, and zram0 is used as swap space.

Creating the New Volume

In this section you will create a new LV in the existing VG, fedora_studentvm1. This is a rather long procedure. If you get interrupted, it is okay to come back to it later. Just be sure to mark which step will be next.

PREPARATION 9: PREPARE A TEST VOLUME

Prepare a new LV for use with these experiments. Start by looking at the current LV configuration. The **lvs** command lists the logical volumes:

```
[root@studentvm1 ~]# lvs
  LV   VG                Attr       LSize  ...
  home fedora_studentvm1 -wi-ao----  2.00g
  root fedora_studentvm1 -wi-ao----  2.00g
  tmp  fedora_studentvm1 -wi-ao----  5.00g
  usr  fedora_studentvm1 -wi-ao---- 15.00g
  var  fedora_studentvm1 -wi-ao---- 10.00g
[root@studentvm1 ~]#
```

And the **vg** command lists the volume group(s) (VG(s)) and some of their statistics. This can show the amount of space left in the VG that is not already assigned to an LV:

```
[root@studentvm1 ~]# vgs
  VG                #PV #LV #SN Attr   VSize   VFree
  fedora_studentvm1   1   5   0 wz--n- <58.00g <24.00g
[root@studentvm1 ~]#
```

From this you can see that there is about 24GB of space left on the VG that has not yet been allocated to an LV. For these experiments we only need a small amount of space, so we will use 500MB for our test volume. Create the new volume and verify that it has been created properly:

```
[root@studentvm1 ~]# lvcreate -L 500M -n test fedora_studentvm1
  Logical volume "test" created.
[root@studentvm1 ~]# lvs
  LV   VG                Attr       LSize   ...
  home fedora_studentvm1 -wi-ao----   2.00g
  root fedora_studentvm1 -wi-ao----   2.00g
  test fedora_studentvm1 -wi-a----- 500.00m
  tmp  fedora_studentvm1 -wi-ao----   5.00g
  usr  fedora_studentvm1 -wi-ao----  15.00g
  var  fedora_studentvm1 -wi-ao----  10.00g
[root@studentvm1 ~]#
```

In the preceding command, -L defines this as a linear volume, 500M is the size, -n test is the name of the new volume, and fedora_studentvm1 is the name of the existing volume group.

The second step in this process is to create an EXT4 filesystem on the new volume. In the following mkfs (make filesystem) command, -t ext4 creates an EXT4 filesystem, and the path is for the device file for this volume. The "mapper" portion of the path is a special location in the device tree that is where LVM devices are managed:

```
[root@studentvm1 ~]# mkfs -t ext4 /dev/mapper/fedora_studentvm1-test
mke2fs 1.46.5 (30-Dec-2021)
```

```
Creating filesystem with 512000 1k blocks and 128016 inodes
Filesystem UUID: 8dfb1594-5d7a-4deb-8cf8-2dd0af0e2a0d
Superblock backups stored on blocks:
        8193, 24577, 40961, 57345, 73729, 204801, 221185, 401409

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

[root@studentvm1 ~]#
```

You can use the following command to verify that the LV has been properly created:

```
[root@studentvm1 ~]# lsblk
NAME                        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                           8:0    0   60G  0 disk
|-sda1                        8:1    0    1M  0 part
|-sda2                        8:2    0    1G  0 part /boot
|-sda3                        8:3    0    1G  0 part /boot/efi
`-sda4                        8:4    0   58G  0 part
  |-fedora_studentvm1-root 253:0    0    2G  0 lvm  /
  |-fedora_studentvm1-usr  253:1    0   15G  0 lvm  /usr
  |-fedora_studentvm1-tmp  253:2    0    5G  0 lvm  /tmp
  |-fedora_studentvm1-var  253:3    0   10G  0 lvm  /var
  |-fedora_studentvm1-home 253:4    0    2G  0 lvm  /home
  `-fedora_studentvm1-test 253:5    0  500M  0 lvm  /test
sr0                          11:0    1 50.5M  0 rom
zram0                       252:0    0    8G  0 disk [SWAP]
[root@studentvm1 ~]#

[root@studentvm1 ~]# mount /dev/sdb1 /test
[root@studentvm1 ~]#
```

You will recall that we added a label to each volume we created during the installation. For the sake of consistency, let's do that now for this new volume. The volume does not need to be mounted for this. First, verify that there is no label, then add the label, and verify that it has been added:

```
[root@studentvm1 ~]# e2label /dev/mapper/fedora_studentvm1-test

[root@studentvm1 ~]# e2label /dev/mapper/fedora_studentvm1-test test
[root@studentvm1 ~]# e2label /dev/mapper/fedora_studentvm1-test
test
[root@studentvm1 ~]#
```

The new volume needs a directory where it can be mounted on the filesystem directory tree. This is called a mount point and is nothing more than a regular directory. Create the /test directory:

```
[root@studentvm1 ~]# mkdir /test
```

You could mount the new test volume manually after every reboot, but that is not the way of the lazy SysAdmin. We can easily add a line to the /etc/fstab (filesystem table)[7] file even without using an editor.[8] The following command-line program appends the required line to /etc/fstab:

```
[root@studentvm1 ~]# echo "/dev/mapper/fedora_studentvm1-test /test ext4  defaults 1 2" >> /etc/fstab
```

Now verify that the new line has been added to the bottom of the file:

```
[root@studentvm1 ~]# cat /etc/fstab
```

The next command tells the system to re-read the fstab file:

```
[root@studentvm1 ~]# systemctl daemon-reload
```

The last step is to mount the new filesystem:

```
[root@studentvm1 ~]# mount /test ; lsblk
NAME                        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                           8:0    0   60G  0 disk
|-sda1                        8:1    0    1M  0 part
|-sda2                        8:2    0    1G  0 part /boot
|-sda3                        8:3    0    1G  0 part /boot/efi
`-sda4                        8:4    0   58G  0 part
  |-fedora_studentvm1-root 253:0    0    2G  0 lvm  /
  |-fedora_studentvm1-usr  253:1    0   15G  0 lvm  /usr
  |-fedora_studentvm1-tmp  253:2    0    5G  0 lvm  /tmp
  |-fedora_studentvm1-var  253:3    0   10G  0 lvm  /var
  |-fedora_studentvm1-home 253:4    0    2G  0 lvm  /home
  `-fedora_studentvm1-test 253:5    0  500M  0 lvm  /test
sr0                          11:0    1 50.5M  0 rom
zram0                       252:0    0    8G  0 disk [SWAP]
[root@studentvm1 ~]#
```

Enter and run the following command-line program to create some files with content on the drive. We use the dmesg command simply to provide data for the files to contain. The contents don't matter so much as just the fact that each file has some content:

```
[root@studentvm1 ~]# cd /test
[root@studentvm1 test]# for I in 0 1 2 3 4 5 6 7 8 9 ; do dmesg > file$I.txt ; done
```

Verify that there are now at least ten files on the drive with the names file0.txt through file9.txt:

```
[root@studentvm1 test]# ll
total 702
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file0.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file1.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file2.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file3.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file4.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file5.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file6.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file7.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file8.txt
-rw-r--r--. 1 root root 69827 Jan 21 16:13 file9.txt
drwx------. 2 root root 12288 Jan 21 10:16 lost+found
[root@studentvm1 test]#
```

The new volume is ready for the experiments in this chapter.

Do not unmount the USB device or detach it from the VM. The test volume is now ready for use in some of the experiments in this chapter.

Generating Data Streams

Most of the core utilities use STDIO as their output stream, and those that generate data streams, rather than acting to transform the data stream in some way, can be used to create the data streams that we will use for our experiments. Data streams can be as short as one line or even a single character and as long as needed.[9]

Let's try our first experiment and create a short data stream.

EXPERIMENT 9-1: GENERATE A SIMPLE DATA STREAM

If you have not done so already, log into the host you are using for these experiments as the user "student." If you have logged into a GUI desktop session, start your favorite terminal emulator; if you have logged into one of the virtual consoles or a terminal emulator, you are ready to go.

Use the command shown in the following to generate a stream of data:

```
[student@studentvm1 test]$ ls -la
total 465229
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file0.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file1.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file2.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file3.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file4.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file5.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file6.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file7.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file8.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file9.txt
drwx------. 2 root root     12288 Jan 21 10:16 lost+found
-rw-r--r--. 1 root root 475674443 Jan 21 17:11 testfile.txt
[root@studentvm1 test]#
```

The output from this command is a short data stream that is displayed on STDOUT, the console or terminal session that you are logged into.

Some GNU core utilities are designed specifically to produce streams of data. Let's take a look at some of these utilities.

EXPERIMENT 9-2: THE YES COMMAND

The **yes** command produces a continuous data stream that consists of repetitions of the data string provided as the argument. The generated data stream will continue until it is interrupted with a Ctrl-C, which is displayed on the screen as ^C.

Enter the command as shown and let it run for a few seconds. Press Ctrl-C when you get tired of watching the same string of data scroll by:

```
[student@studentvm1 test]$ yes 123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
123465789-abcdefg
1234^C
```

Now just enter the yes command with no options:

```
[student@studentvm1 test]$ yes
y
y
y
y
y
y^C
```

The primary function of the **yes** command is to produce a stream of data.

"What does this prove?" you ask. Just that there are many ways to create a data stream that might be useful. When run as root, the **rm *** command will erase every file in the present working directory (PWD) – but it asks you to enter "y" for each file to verify that you actually want to delete that file.[10] This means more typing.

EXPERIMENT 9-3: USING YES AS INPUT TO COMMANDS

Perform this experiment as the root user in using /test as the PWD.

I haven't talked about pipes yet, but as a SysAdmin, or someone who wants to become one, you should know how to use them. The following CLI program will supply the response of "y" to each request by the rm command and will delete all of the files in the PWD.

Start by trying to delete all of the files in the /test directory. Ensure that /test is the PWD:

```
[root@studentvm1 test]# rm file*txt
rm: remove regular file 'file0.txt'?
```

Press Ctrl-C[11] to exit this command, which will ask for a separate permission to delete each file. This can be a bit time consuming for you when there are very large numbers of files.

The following command-line program provides the necessary input to the **rm** command, so no further intervention is required by you:

```
[root@studentvm1 test]# yes | rm file*txt ; ll
rm: remove regular file 'file0.txt'? rm: remove regular file 'file1.txt'? rm: remove regular file 'file2.txt'? rm: remove regular file 'file3.txt'? rm: remove
regular file 'file4.txt'? rm: remove regular file 'file5.txt'? rm: remove regular file 'file6.txt'? rm: remove regular file 'file7.txt'? rm: remove regular
file 'file8.txt'? rm: remove regular file 'file9.txt'? total 0
[root@studentvm1 test]#  ll
total 12
drwx------. 2 root root 12288 Jan 21 10:16 lost+found
[root@studentvm1 test]#
```

Warning! Do not run this command anywhere but the /test location as specified in this experiment because it will delete all of the files in the PWD.

Now recreate the files we just deleted using the seq (sequence) command to generate the file numbers instead of providing them as a list as we did previously. Then verify that the files have been recreated:

```
[root@studentvm1 test]# for I in `seq 0 9` ; do dmesg > file$I.txt ; done ; ll
```

You can also use **rm -f f*t**, which would also forcibly delete all of the files in the PWD. The -f means "force" the deletions. Be sure you are in the /test directory where the USB device is mounted. Then run the following commands to delete all the files we just created, and verify they are gone:

```
rm -f f*t ; ll
```

This is something you should not do without ensuring that the files really should be deleted.

Once more, recreate the test files in /test. Note that you can save some time using command-line recall. Simply press the up arrow key to scroll back through the previous commands until you get to the one you want. Then press the Enter key.

Do not unmount the USB device.

Test a Theory with Yes

Another option for using the yes command is to fill a directory with a file containing some arbitrary and irrelevant data in order to – well – fill up the directory. I have used this technique to test what happens to a Linux host when a particular directory becomes full. In the specific instance where I used this technique, I was testing a theory because a customer was having problems and could not log into their computer.

EXPERIMENT 9-4: TESTING WITH YES

This experiment should be performed as root.

In order to prevent filling the root filesystem, this experiment will use the test volume that you should have prepared in advance in the "Preparing a Logical Volume for Testing" section of this chapter. This experiment will not affect the existing files on that volume.

Make sure that /test is the PWD.

Let's take the time to learn two more tools, the **watch** utility, which works nicely to make a static command such as **df** into one that continuously updates. The **df** utility displays the filesystems, their sizes, free space, and mount points. Just run the **df** command first to see what that looks like:

```
[root@studentvm1 test]# df
Filesystem                         1K-blocks    Used Available Use% Mounted on
devtmpfs                                4096       0      4096   0% /dev
tmpfs                                8190156      12   8190144   1% /dev/shm
tmpfs                                3276064    1192   3274872   1% /run
/dev/mapper/fedora_studentvm1-root   1992552   39600   1831712   3% /
/dev/mapper/fedora_studentvm1-usr   15375304 4817352   9755136  34% /usr
/dev/sda2                             996780  233516    694452  26% /boot
/dev/sda3                            1046508   17804   1028704   2% /boot/efi
/dev/mapper/fedora_studentvm1-tmp    5074592     392   4795672   1% /tmp
/dev/mapper/fedora_studentvm1-home   1992552   29716   1841596   2% /home
/dev/mapper/fedora_studentvm1-var   10218772  404012   9274088   5% /var
tmpfs                                1638028      88   1637940   1% /run/user/1000
tmpfs                                1638028      64   1637964   1% /run/user/0
/dev/mapper/fedora_studentvm1-test    469328     704    438928   1% /test
[root@studentvm1 test]#
```

The -h option presents the numbers in (h)uman-readable format:

```
[root@studentvm1 test]# df -h
Filesystem                         Size  Used Avail Use% Mounted on
devtmpfs                           4.0M     0  4.0M   0% /dev
tmpfs                              7.9G   12K  7.9G   1% /dev/shm
tmpfs                              3.2G  1.2M  3.2G   1% /run
/dev/mapper/fedora_studentvm1-root  2.0G   39M  1.8G   3% /
/dev/mapper/fedora_studentvm1-usr    15G  4.6G  9.4G  34% /usr
/dev/sda2                          974M  229M  679M  26% /boot
/dev/sda3                         1022M   18M 1005M   2% /boot/efi
/dev/mapper/fedora_studentvm1-tmp   4.9G  392K  4.6G   1% /tmp
/dev/mapper/fedora_studentvm1-home  2.0G   30M  1.8G   2% /home
/dev/mapper/fedora_studentvm1-var   9.8G  395M  8.9G   5% /var
tmpfs                              1.6G   88K  1.6G   1% /run/user/1000
tmpfs                              1.6G   64K  1.6G   1% /run/user/0
/dev/mapper/fedora_studentvm1-test  459M  704K  429M   1% /test
[root@studentvm1 test]#
```

Note that the default units for the df command are 1K blocks. In one root terminal session, start the watch command and use the df command as its argument. This constantly updates the disk usage information and allows us to watch as the USB device fills up. The -n option on the watch command tells it to run the df command every one second instead of the default two seconds. That looks like this:

```
[root@studentvm1 ~]# watch -n 1 df
Every 1.0s: df                                      studentvm1: Fri Jan 20 16:06:11 2023

Filesystem                         1K-blocks    Used Available Use% Mounted on
devtmpfs                                4096       0      4096   0% /dev
tmpfs                                2006112       0   2006112   0% /dev/shm
tmpfs                                 802448    1244    801204   1% /run
/dev/mapper/fedora_studentvm1-root   1992552   25932   1845380   2% /
/dev/mapper/fedora_studentvm1-usr   15375304 4819656   9752832  34% /usr
/dev/mapper/fedora_studentvm1-home   1992552   29168   1842144   2% /home
/dev/mapper/fedora_studentvm1-tmp    5074592     228   4795836   1% /tmp
/dev/mapper/fedora_studentvm1-var   10218772  341856   9336244   4% /var
/dev/sda2                             996780  233516    694452  26% /boot
/dev/sda3                            1046508   17804   1028704   2% /boot/efi
tmpfs                                 401220      88    401132   1% /run/user/1000
tmpfs                                 401220      64    401156   1% /run/user/0
/dev/sdb1                            7812864      16   7812848   1% /test
```

The data will update every second.

Place this terminal session somewhere on your desktop so that you can see it; then, as root open another terminal session and run the command shown in the following. Depending upon the size of your USB filesystem, the time to fill it may vary, but it should be quite fast on a small-capacity test volume. The first time I tested this, it took 18 minutes and 55 seconds on my system with a 4GB USB device.

Note that we are redirecting a long data stream. Watch the /dev/sdb1 filesystem on /test as it fills up:

```
[root@studentvm1 test]# yes 123456789-abcdefgh >> /test/testfile.txt
yes: standard output: No space left on device
[root@studentvm1 test]#
```

When the filesystem fills up, the error is displayed and the program terminates. The **df** command output should look like this:

```
Filesystem                        1K-blocks    Used Available Use% Mounted on
devtmpfs                               4096       0      4096   0% /dev
tmpfs                               8190156      12   8190144   1% /dev/shm
tmpfs                               3276064    1200   3274864   1% /run
/dev/mapper/fedora_studentvm1-root   1992552   39600   1831712   3% /
/dev/mapper/fedora_studentvm1-usr   15375304 4817352   9755136  34% /usr
/dev/sda2                            996780  233516    694452  26% /boot
/dev/sda3                           1046508   17804   1028704   2% /boot/efi
/dev/mapper/fedora_studentvm1-tmp    5074592     392   4795672   1% /tmp
/dev/mapper/fedora_studentvm1-home   1992552   29716   1841596   2% /home
/dev/mapper/fedora_studentvm1-var   10218772  404024   9274076   5% /var
tmpfs                               1638028      88   1637940   1% /run/user/1000
tmpfs                               1638028      64   1637964   1% /run/user/0
/dev/mapper/fedora_studentvm1-test    469328  465231         0 100% /test

[root@studentvm1 test]# ll
total 465229
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file0.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file1.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file2.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file3.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file4.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file5.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file6.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file7.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file8.txt
-rw-r--r--. 1 root root     69827 Jan 21 16:53 file9.txt
drwx------. 2 root root     12288 Jan 21 10:16 lost+found
-rw-r--r--. 1 root root 475674443 Jan 21 17:11 testfile.txt
[root@studentvm1 test]#
```

Your results should look similar to mine. Be sure to look at the line from the **df** output that refers to the /test volume. This shows that 100% of the space on that filesystem is used.

Now delete testfile.txt from /test:

```
[root@studentvm1 ~]# rm -f /test/testfile.txt
```

I used the simple test in Experiment 9-4 on the /tmp directory of one of my own computers as part of my testing to assist me in determining my customer's problem. After /tmp filled up, users were no longer able to log into a GUI desktop, but they could still log in using the consoles. That is because logging into a GUI desktop creates new files in the /tmp directory, and there was no room left so the login failed. The console login does not create new files in /tmp, so they succeeded. My customer had not tried logging into the console because they were not familiar with the CLI.

After testing this on my own system as verification, I used the console to log into the customer host and found a number of large files taking up all of the space in the /tmp directory. I deleted those and helped the customer determine how the files were being created, and we were able to put a stop to that.

The Boot Record

It is now time to do a little exploring, and to be as safe as possible, you will – mostly – use the test volume that you have already been experimenting with. In this experiment we will look at some of the filesystem structures.

Let's start with something simple, the **dd** command. Officially known as "disk dump," many SysAdmins call it "disk destroyer" for good reason. Many of us have inadvertently destroyed the contents of an entire hard drive or partition using the **dd** command. That is why we will use the test volume to perform some of these experiments.

Despite its reputation, **dd** can be quite useful in exploring various types of storage media, storage devices, and partitions. We will also use it as a tool to explore other aspects of Linux.

EXPERIMENT 9-5: EXPLORING THE BOOT RECORD

This experiment must be performed as root. Log into a terminal session as root if you are not already.

The boot record is a single block of data located at the beginning of every storage device. Having installed Linux on the only HDD on the VM, /dev/sda, the boot record is the first sector of that device. The boot record is not located in any partition or logical volume.

As root in a terminal session, look at the block (HDD or SSD) devices on the VM:

```
[root@studentvm1 test]# lsblk
NAME                       MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                          8:0    0   60G  0 disk
|-sda1                       8:1    0    1M  0 part
|-sda2                       8:2    0    1G  0 part /boot
|-sda3                       8:3    0    1G  0 part /boot/efi
`-sda4                       8:4    0   58G  0 part
  |-fedora_studentvm1-root 253:0    0    2G  0 lvm  /
  |-fedora_studentvm1-usr  253:1    0   15G  0 lvm  /usr
  |-fedora_studentvm1-tmp  253:2    0    5G  0 lvm  /tmp
  |-fedora_studentvm1-var  253:3    0   10G  0 lvm  /var
  |-fedora_studentvm1-home 253:4    0    2G  0 lvm  /home
  `-fedora_studentvm1-test 253:5    0  500M  0 lvm  /test
sr0                         11:0    1 50.5M  0 rom
zram0                      252:0    0    8G  0 disk [SWAP]
```

Use the dd command to view the boot record of the virtual hard drive, /dev/sda. The bs= argument is not what you might think; it simply specifies the block size. And the count= argument specifies the number of blocks to dump to STDIO. The if= (input file) argument specifies the source of the data stream, in this case the test volume:

```
[root@studentvm1 test]# dd if=/dev/sda bs=1024 count=1
ʿcʿʿʿʿʿʿtʿʿptʿʿʿy|1ʿʿـهM ʿʿd|<ʿtʿʿRʿʿ}ʿʿ|ʿAʿʿUʿZRr=ʿʿUʿu7ʿʿt21ʿʿD@ʿDʿʿDʿfʿ\|fʿfʿ`|fʿ\
Zʿʿʿʿʿʿ}ʿʿfʿcdʿ@ʿDʿʿʿʿʿʿʿʿ@ʿʿʿʿʿʿʿʿ`|f         ʿuNfʿ\|f1ʿfʿ4ʿʿ1ʿfʿt;}7ʿʿʿʿ0ʿʿʿʿʿZʿƻpʿʿ1ʌʿrʿʿ`ʿʿʿ1ʿʿʿʿʿʿʿʿaʿ&Z|ʿʿ}ʿʿʿ}ʿ4ʿʿ}ʿ.ʿ▒ʿʿGRUB GeomHard DiskRead Error
ʿʿʿʿ<uʿʿʿʿʿʿʿʿʿUʿEFI PART\(`*ʿʿ"ʿʿʿ      @ʿT
Fʿʿʿʿ1>ʿʿʿ~Jʿ
1+0 records in
1+0 records out
1024 bytes (1.0 kB, 1.0 KiB) copied, 0.00160644 s, 637 kB/s
[root@studentvm1 test]#
```

This prints the text of the boot record, which is the first block on the storage device – any HDD or SSD. In this case, there is information about the filesystem and, although it is unreadable because it is stored in binary format, the partition table. Since this is a bootable device, stage 1 of GRUB or some other boot loader is located in this sector. The last three lines contain data about the number of records and bytes processed.

Now do the same experiment, but on the first record of the first partition.

EXPERIMENT 9-6: LOOKING AT A PARTITION RECORD

Run the following command as root. This looks at the first record in the first partition of the /dev/sda device, /dev/sda1:

```
[root@studentvm1 test]# dd if=/dev/sda1 bs=1024 count=1
RVʿʿʿ9^ʿʿfʿ-ʿʿʿʿ|ʿtFʿʿMf1ʿ9ʿ)fʿUʿʿDfʿfʿLʿDpPʿDʿBʿʿʿʿpʿffʿEf  ʿʿʿfʿf1ʿfʿ4ʿT
f1ʿfʿtʿT
        ʿD;}yʿ*D
ʿʿʿLfʿUʿT
ʿъlZRʿt
       Pʿpʿʿ1ɾʿrFʿÎE
ʿʿE
`ʿʿʿʿ1ʿ1ʿʿʿʿʿ#ʿʿWaʿʿ$ʿʿʿʿʿʿ%ʿʿBZʿʿʿʿʿ6ʿʿ-ʿʿ.ʿ2ʿʿʿʿloading.
 ^

Бʿʿdʿʿ$ʿʿʿʿʿĉʿfʿ  ʿـهʿʿʿʿʿiʿ  ʿʿʿʿ"ʿfʿ{ʿf1ʿـهʿʿʿʿʿfʿUʿʿWʿʿV1ʿSʿÉʿ1ʿx+ʿʿʿtʿʿʿʿʿʿʿdʿʿĉšʿʿ$1ʿ
1ʿʿʿʿ~ʿʿʿHʿʿ[ʿʿ^_]Äʿt ʿʿtʿʿʿʿʿʿʿʿʿʿʿ1ʿʿUʿʿWVSʿʿ1+0 records in
1+0 records out
1024 bytes (1.0 kB, 1.0 KiB) copied, 0.66347 s, 1.5 kB/s
[root@studentvm1 test]#
```

This experiment shows the that there are differences between a boot record and the first record of a partition. It also shows that the **dd** command can be used to view data in the partitions as well as for the disk itself.

What else is out there on the test volume? Depending upon the specifics of the USB device you are using for these experiments, you may have somewhat different results from mine. I will show you what I did, and you can modify that if necessary to achieve the desired result.

What we are attempting to do is use the **dd** command to locate the directory entries for the files we created on the test volume and then some of the data. If we had enough knowledge of the metadata structures, we could interpret them directly to find the locations of this data on the drive, but we don't so we will have to do this the hard way – print out data until we find what we want.

So let's start with what we do know and proceed with a little finesse. We know that the data files we created during the LV preparation were in the first partition on the device. Therefore, we don't need to search the space between the boot record and the first partition, which contains lots of emptiness. At least that is what it should contain.

Starting with the beginning of /dev/sda1, let's look at a few blocks of data at a time to find what we want. The command in Experiment 9-7 is similar to the previous one except that we specify a few more blocks of data to view. You may have to specify fewer blocks if your terminal is not large enough to display all of the data at one time, or you can pipe the data through the less utility and use that to page through the data. Either way works. Remember we are doing all of this as root user because non-root users do not have the required permissions.

Digging Deeper

There is far more to storage devices than only the boot record. So let's look further into the partitions on the storage device sda.

EXPERIMENT 9-7: DISPLAY MANY RECORDS OF A PARTITION

Enter the same command as you did in the previous experiment, but this time use /dev/sda4 and increase the block count to be displayed to 2000 as shown in the following in order to show more data.

Note that "^@^@^@^@" is essentially null data. There can be a lot of it between the first record of the partition and the beginning of the data area:

```
[root@studentvm1 ~]# dd if=/dev/sda4 bs=512 count=2000 | less
```

Page down until you see something like this:

```
^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@
{
id = "KUZ1r9-FGf5-QXR1-xvx9-8gff-N85k-sicFij"
seqno = 1
format = "lvm2"
```

```
status = ["RESIZEABLE", "READ", "WRITE"]
flags = []
extent_size = 8192
max_lv = 0
max_pv = 0
metadata_copies = 0

physical_volumes {

pv0 {
id = "czfgYd-e5tH-b4PK-2PXd-74Tx-9vkC-SWduDi"
device = "/dev/sda4"

status = ["ALLOCATABLE"]
flags = []
dev_size = 121628672
pe_start = 2048
pe_count = 14847
}
```

This data is part of the metadata for the volume group. There is quite a bit of LVM metadata so it will take a while to scroll through it. Scroll down some more until you see something like this. This is the LVM metadata for the /home volume. All of the volume definitions are located in this area of the sda4 device:

```
home {
id = "sfKPPQ-7kAx-fIdF-S74h-St3W-Ppw3-HfNhlZ"
status = ["READ", "WRITE", "VISIBLE"]
flags = []
creation_time = 1673958577
creation_host = "localhost-live"
segment_count = 1

segment1 {
start_extent = 0
extent_count = 512

type = "striped"
stripe_count = 1

stripes = [
"pv0", 3840
]
}
}
```

It can take a long time to scroll through this data, but we do have another option.

Let's look at a new option for the `dd` command, one which gives us a little more flexibility.

EXPERIMENT 9-8: STARTING AT OTHER THAN THE FIRST RECORD

We now want to display 100 blocks of data at a time, but we don't want to start at the beginning of the partition, we want to skip the blocks we have already looked at.

Enter the following command and add the `skip` argument, which skips the first 2000 blocks of data and displays the next 100:

```
[root@studentvm1 test]# dd if=/dev/sda4 bs=512 count=100 skip=2000
10+0 records in
10+0 records out
5120 bytes (5.1 kB, 5.0 KiB) copied, 0.01786 s, 287 kB/s
```

This set of parameters may not display the file data for you if your test volume is a different size or is formatted differently, but it should be a good place to start. You can continue iterating until you find the data. You should definitely take some time on your own to explore the contents of the other partitions. You might be surprised at what you find.

Randomness

It turns out that randomness is a desirable thing in computers. Who knew. There are a number of reasons that SysAdmins might want to generate a stream of random data. A stream of random data is sometimes useful to overwrite the contents of a complete partition, such as /dev/sda1, or even the entire hard drive as in /dev/sda.

Although deleting files may seem permanent, it is not. Many forensic tools are available and can be used by trained specialists to easily recover files that have supposedly been deleted. It is much more difficult to recover files that have been overwritten by random data. I have frequently needed not just to delete all of the data on a hard drive but to overwrite it so it cannot be recovered. I do this for customers and friends who have "gifted" me with their old computers for reuse or recycling.

Regardless of what ultimately happens to the computers, I promise the people who donate the computers that I will scrub all of the data from the hard drive. I remove the drives from the computer, put them in my plugin hard drive docking station, and use a command similar to the one in Experiment 9-9 to overwrite all of the data, but instead of just spewing the random data to STDOUT as in this experiment, I redirect it to the device file for the hard drive that needs to be overwritten – but don't do that.

EXPERIMENT 9-9: GENERATING RANDOMNESS

Perform this experiment as the student user. Enter this command to print an unending stream of random data to STDIO:

```
[student@studentvm1 test]$ cat /dev/urandom
```

Use **Ctrl-C** to break out and stop the stream of data. You may need to use **Ctrl-C** multiple times.

If you are extremely paranoid, the **`shred`** command can be used to overwrite individual files as well as partitions and complete drives. It can write over the device as many times as needed for you to feel secure, with multiple passes using both random data and specifically sequenced patterns of data designed to prevent even the most sensitive equipment from recovering any data from the hard drive. As with other utilities that use random data, the random stream is supplied by the /dev/urandom device.

Random data is also used as the input seed to programs that generate random passwords and random data and numbers for use in scientific and statistical calculations. We will cover randomness and other interesting data sources in a bit more detail in Volume 2, Chapter 22.

Pipe Dreams

Pipes are critical to our ability to do the amazing things on the command line, so much so that I think it is important to recognize that they were invented by Douglas McIlroy[12] during the early days of Unix. Thanks, Doug! The Princeton University website has a fragment of an interview[13] with McIlroy in which he discusses the creation of the pipe and the beginnings of the Unix Philosophy.

Notice the use of pipes in the simple command-line program shown in Experiment 9-10 that lists each logged-in user a single time no matter how many logins they have active.

EXPERIMENT 9-10: INTRODUCING PIPES

Perform this experiment as the student user. Enter the command shown in the following:

```
[student@studentvm1 test]$ w | tail -n +3 | awk '{print $1}' | sort | uniq
root
student
[student@studentvm1 test]$
```

The results from this command produce two lines of data that show that the users root and student are both logged in. It does not show how many times each user is logged in.

Pipes – represented by the vertical bar (|) – are the syntactical glue, the operator, that connects these command-line utilities together. Pipes allow the standard output from one command to be "piped," that is, streamed from the standard output of one command to the standard input of the next command.

The *|&* operator can be used to pipe STDERR along with STDOUT to STDIN of the next command. This is not always desirable, but it does offer flexibility in the ability to record the STDERR data stream for the purposes of problem determination.

A string of programs connected with pipes is called a pipeline.

Think about how this program would have to work if we could not pipe the data stream from one command to the next. The first command would perform its task on the data, and then the output from that command would have to be saved in a file. The next command would have to read the stream of data from the intermediate file and perform its modification of the data stream, sending its own output to a new, temporary data file. The third command would have to take its data from the second temporary data file and perform its own manipulation of the data stream and then store the resulting data stream in yet another temporary file. At each step the data file names would have to be transferred from one command to the next in some way.

I cannot even stand to think about that because it is so complex. Remember that simplicity rocks!

Building Pipelines

When I am doing something new, solving a new problem, I usually do not just type in a complete bash command pipeline from scratch, as in Experiment 9-10 off the top of my head. I usually start with just one or two commands in the pipeline and build from there by adding more commands to further process the data stream. This allows me to view the state of the data stream after each of the commands in the pipeline and make corrections as they are needed.

In Experiment 9-11 you should enter the command shown on each line and run it as shown to see the results. This will give you a feel for how you can build up complex pipelines in stages.

EXPERIMENT 9-11: BUILDING A PIPELINE

Enter the commands as shown on each line. Observe the changes in the data stream as each new filter utility is inserted to the data stream using the pipe.

Log in as root to two of the Linux virtual consoles and as the student user to two additional virtual consoles, and open several terminal sessions on the desktop. This should give plenty of data for this experiment:

```
[student@studentvm1 test]$ w
[student@studentvm1 test]$ w | tail -n +3
[student@studentvm1 test]$ w | tail -n +3 | awk '{print $1}'
[student@studentvm1 test]$ w | tail -n +3 | awk '{print $1}' | sort
[student@studentvm1 test]$ w | tail -n +3 | awk '{print $1}' | sort | uniq
```

The results of this experiment illustrate the changes to the data stream performed by each of the filter utility programs in the pipeline.

It is possible to build up very complex pipelines that can transform the data stream using many different utilities that work with STDIO.

Redirection

Redirection is the capability to redirect the STDOUT data stream of a program to a file instead of to the default target of the display. The "greater than" (>) character, a.k.a. "gt", is the syntactical symbol for redirection. Experiment 9-12 shows how to redirect the output data stream of the `df -h` command to the file diskusage.txt.

EXPERIMENT 9-12: REDIRECTING STDOUT

Redirecting the STDOUT of a command can be used to create a file containing the results from that command:

```
[student@studentvm1 test]$ df -h > diskusage.txt
```

There is no output to the terminal from this command unless there is an error. This is because the STDOUT data stream is redirected to the file and STDERR is still directed to the STDOUT device, which is the display. You can view the contents of the file you just created using this next command:

```
[student@studentvm1 test]$ cat diskusage.txt
Filesystem                          Size  Used Avail Use% Mounted on
devtmpfs                            2.0G     0  2.0G   0% /dev
tmpfs                               2.0G     0  2.0G   0% /dev/shm
tmpfs                               2.0G  1.2M  2.0G   1% /run
tmpfs                               2.0G     0  2.0G   0% /sys/fs/cgroup
/dev/mapper/fedora_studentvm1-root  2.0G   49M  1.8G   3% /
/dev/mapper/fedora_studentvm1-usr    15G  3.8G   11G  27% /usr
/dev/sda1                           976M  185M  724M  21% /boot
/dev/mapper/fedora_studentvm1-tmp   4.9G   21M  4.6G   1% /tmp
/dev/mapper/fedora_studentvm1-var   9.8G  504M  8.8G   6% /var
/dev/mapper/fedora_studentvm1-home  2.0G  7.3M  1.8G   1% /home
tmpfs                               395M  8.0K  395M   1% /run/user/1000
tmpfs                               395M     0  395M   0% /run/user/0
/dev/sdb1                            60M  440K   59M   1% /test
[student@studentvm1 test]$
```

When using the > symbol for redirection, the specified file is created if it does not already exist. If it already does exist, the contents are overwritten by the data stream from the command. You can use double greater than symbols, >>, to append the new data stream to any existing content in the file as illustrated in Experiment 9-13.

EXPERIMENT 9-13: APPENDING REDIRECTED DATA STREAMS
This command appends the new data stream to the end of the existing file:

```
[student@studentvm1 test]$ df -h >> diskusage.txt
```

You can use cat and/or less to view the diskusage.txt file in order to verify that the new data was appended to the end of the file.

The < (less than) symbol redirects data to the STDIN of the program. You might want to use this method to input data from a file to STDIN of a command that does not take a file name as an argument but that does use STDIN. Although input sources can be redirected to STDIN, such as a file that is used as input to **grep**, it is generally not necessary as **grep** also takes a file name as an argument to specify the input source. Most other commands also take a file name as an argument for their input source.

One example of using redirection to STDIN is with the **od** command as shown in Experiment 9-14. The -N 50 option prevents the output from continuing forever. You could use Ctrl-C to terminate the output data stream if you don't use the -N option to limit it.

EXPERIMENT 9-14: REDIRECTING STDIN
This experiment illustrates the use of redirection as input to STDIN:

```
[student@studentvm1 test]$ od -c -N 50 < /dev/urandom
0000000  331  203    _  307    ]    {  335  337    6  257  347         $    J    Z    U
0000020  245   \0    `   \b    8  307  261  207    K    :    }    S    \  276  344    ;
0000040  336  256  221  317  314  241  352    `  253  333  367  003  374  264  335    4
0000060    U   \n  347    (    h  263  354  251    u    H    ]  315  376    W  205   \0
0000100  323  263  024    %  355  003  214  354  343    \    a  254    #    `    {    _
0000120    b  201  222    2  265    [  372  215  334  253  273  250    L    c  241  233
<snip>
```

It is much easier to understand the nature of the results when formatted using **od** (Octal Display), which formats the data stream in a way that is a bit more intelligible. Read the man page for **od** for more information.

Redirection can be the source or the termination of a pipeline. Because it is so seldom needed as input, redirection is usually used as termination of a pipeline.

EXPERIMENT 9-15: USING ECHO TO GENERATE TEXT STREAMS
Perform this experiment as the student user. This activity provides examples of some aspects of redirection not yet covered. The **echo** command is used to print text strings to STDOUT.

Make your home directory the PWD and create a small text file:

```
[student@studentvm1 test]$ echo "Hello world" > hello.txt
```

Read the contents of the file by redirecting it to STDIN:

```
[student@studentvm1 test]$ cat < hello.txt
Hello world
[student@studentvm1 test]$
```

Add another line of text to the existing file:

```
[student@studentvm1 test]$ echo "How are you?" >> hello.txt
```

View the contents:

```
[student@studentvm1 test]$ cat hello.txt
Hello world
How are you?
[student@studentvm1 test]$
```

Delete (remove) the file and list the contents of your home directory to verify that the file has been erased:

```
[student@studentvm1 test]$ rm hello.txt ; ls -l
```

Create the file again:

```
[student@studentvm1 test]$ echo "Hello world" >> hello.txt ; ll
```

Verify that the file was recreated using the ls and cat commands:

Note that in this last case, the >> operator created the file because it did not exist. If it has already existed, the line would have been added at the end of the existing file as it was in step 4. Also notice the quotes are standard ASCII quotes, the same before and after the quoted string, and not extended ASCII, which are different before and after.

Just grepping Around

The **grep** command is used to select lines that match a specified pattern from a stream of data. **grep** is one of the most commonly used filter utilities and can be used in some very creative and interesting ways. The **grep** command is one of the few that can correctly be called a filter because it does filter out all the lines of the data stream that you do not want; it leaves only the lines that you do want in the remaining data stream.

According to Klaatu, my reviewer for Volume 3 of this course, "One of the classic Unix commands, developed way back in 1974 by Ken Thompson, is the Global Regular Expression Print (grep) command. It's so ubiquitous in computing that it's frequently used as a verb ('grepping through a file') and, depending on how geeky your audience, it fits nicely into real-world scenarios, too. (For example, 'I'll have to grep my memory banks to recall that information.') In short, grep is a way to search through a file for a specific pattern of characters. If that sounds like the modern Find function available in any word processor or text editor, then you've already experienced grep's effects on the computing industry."[14]

EXPERIMENT 9-16: INTRODUCING GREP
We need to create a file with some random data in it. We can use a tool that generates random passwords, but we first need to install it as root:

```
dnf -y install pwgen
```

Now as the student user, let's generate some random data and create a file with it. If the PWD is not /test, make it so. The following command creates a stream of 5000 lines of random data that are each 75 characters long and stores them in the random.txt file:

```
pwgen 75 5000 > random.txt
```

Considering that there are so many passwords, it is very likely that some character strings in them are the same. Use the grep command to locate some short, randomly selected strings from the last ten passwords on the screen. I saw the words "see" and "loop" in one of those ten passwords, so my command looked like this:

```
grep see random.txt
```

You can try that, but you should also pick some strings of your own to check. Short strings of two to four characters work best.

Use the grep filter to locate all of the lines in the output from dmesg with CPU in them:

```
dmesg | grep cpu
```

List all of the directories in your home directory with the command

```
ls -la | grep ^d
```

This works because each directory has a "d" as the first character in a long listing. The caret (^) is used by grep and other tools to anchor the text being searched to the beginning of the line.

To list all of the files that are not directories, reverse the meaning of the previous grep command with the -v option:

```
ls -la | grep -v ^d
```

Chapter Summary

You probably noticed that this chapter was not just about creating random data streams. You created meaningful data streams and then manipulated them to view or use specific data. Much of that data relates to storage because you need an LV for more experiments in later chapters, and this is a good opportunity to explore both data streams and disk and LVM structure.

It is only with the use of pipes, pagers, and redirection that many of the amazing and powerful tasks that can be performed on the Linux command line are possible. It is the pipes that transport STDIO data streams from one program or file to another. In this chapter you have learned that piping streams of data through one or more filter programs supports powerful and flexible manipulation of data in those streams.

Each of the programs in the pipelines demonstrated in the experiments is small, and each does one thing well. They are also filters, that is, they take the standard input, process it in some way, and then send the result to the standard output. Implementation of these programs as filters to send processed data streams from their own standard output to the standard input of the other programs is complementary to and necessary for the implementation of pipes as a Linux tool.

You also learned that STDIO is nothing more than streams of data. This data can be almost anything from the output of a command to list the files in a directory to an unending stream of data from a special device like /dev/urandom or even a stream that contains all of the raw data from a hard drive, an LV, or a partition. You learned some different and interesting methods to generate different types of data streams and how to use the **dd** command to explore the contents of a hard drive.

Any device on a Linux computer can be treated like a data stream. You can use ordinary tools like **dd** and **cat** to dump data from a device into a STDIO data stream that can be processed using other ordinary Linux tools.

Exercises

Do the following exercises to complete this chapter:

1. What is the function of the greater than symbol (>)?
2. Is it possible to append the content of a data stream to an existing file?
3. Design a short command-line program to display the line containing the CPU model name and nothing else.
4. Create a file in the /test directory that consists of ten lines of random data.

Footnotes

1 In Linux systems all hardware devices are treated as files. More about this in Chapter 22, Volume 2.

2 Raymond, Eric S., *The Art of Unix Programming*, www.catb.org/esr/writings/taoup/html/ch01s06.html

3 Linuxtopia, *Basics of the Unix Philosophy*, www.linuxtopia.org/online_books/programming_books/art_of_unix_programming/ch01s06.html

4 Wikipedia, *Ken Thompson*, https://en.wikipedia.org/wiki/Ken_Thompson

5 Type 83 is a standard Linux partition. Linux supports many different partitions. Filesystems and types are covered in Chapter 19 of this volume.

6 LVM is covered in Volume 2, Chapter 20.

7 The /etc/fstab file is explored in detail in Chapter 19 in this volume.

8 Editors are coming up in the next chapter.

9 A data stream taken from special device files random, urandom, and zero, for example, can continue forever without some form of external termination such as the user entering Ctrl-c, a limiting argument to the command, or a system failure.

10 The -f option to the **rm** command forces the **rm** command to delete all files without asking the user. But this experiment is a good illustration of the use of the yes command.

11 The Ctrl-C key combination kills the process. Press and hold the Ctrl key and then press the C key.

12 Wikipedia, *Biography of Douglas McIlroy*, www.cs.dartmouth.edu/~doug/biography

13 Princeton University, *Interview with Douglas McIlroy*, www.princeton.edu/~hos/frs122/precis/mcilroy.htm

14 Kenlon, Seth, a.k.a. Klaatu, Opensource.com, *Practice using the Linux grep command*, March 18, 2021